



**VNiVERSiDAD
D SALAMANCA**
Campus de Excelencia Internacional

EcoSalmántica Smart City

Plataforma integral de servicios municipales

Administración de Sistemas

Autores:

Carlos Cuadrado Pérez-Rubín

Sergio Hernández Moya

Profesor:

Gabriel Villarrubia Gonzalez

Curso:

2026

Índice

EcoSalmántica Smart City

1. Introducción.....	2
2. Configuración inicial del servidor.....	2
3. Configuración de Apache y plataforma web.....	3
3.1. Configuración inicial de Apache y HTTPS.....	3
3.2. Desarrollo CGI, autenticación y gestión web.....	6
3.2.1. Activación de CGI y gestión de sesiones.....	6
3.2.2. Autenticación mediante pwauth.....	7
3.2.3. Registro y activación de usuarios.....	9
3.2.4. Gestión de documentación técnica.....	13
3.2.5 Acceso remoto mediante SFTP.....	15
3.2.6 Recuperación de la contraseña.....	16
3.2.7 Cambios en el perfil del usuario.....	18
3.2.7.1 Modificación de datos personales, cambiar_datos.pl.....	18
3.2.7.2 Actualizar contraseña, cambiar_password.pl.....	19
3.2.7.3 Baja y destrucción de la sesión.....	19
3.2.8 Mantenimiento y recolección de basura.....	20
3.2.9 Dashboard dinámico.....	22
3.2.10 Cron.....	23
3.2.11 Configuración notificaciones externas.....	25
3.3. Blog ciudadano y participación web.....	26
4. Monitorización y estadísticas.....	28
4.1. Instalación de Docker, Grafana y Prometheus.....	28
4.2. Grafana y paneles de monitorización.....	29
4.3. Informes automáticos de monitorización.....	31
5. Sistema de copias de seguridad.....	35
6. Cuotas de disco y gestión de almacenamiento.....	37
7. Seguridad y registros del sistema.....	39
7.1. Registro de accesos SSH/SFTP.....	39
7.2. Registro de accesos web.....	41
8. Base de datos Mariadb.....	42
8.1 Instalación de la base de datos.....	42
8.2 Creación nueva base de datos.....	42
9. Servidor de correo.....	44
9.1 Postfix.....	44
9.2 Dovecot.....	45
9.3 Roundcube.....	45
10. Mejoras y funcionalidades extra.....	46
10.1 Banner SSH.....	46
10.2. Clasificación automática de incidencias mediante IA.....	47
11. Conclusiones.....	51

1. Introducción

EcoSalmántica Smart City es una plataforma desarrollada sobre un servidor Ubuntu Server, distribución basada en Debian GNU/Linux, cuyo objetivo es centralizar diferentes servicios municipales en un único sistema. La plataforma permite gestionar usuarios ciudadanos y técnicos, ofreciendo servicios como portal web, correo electrónico, blogs, almacenamiento remoto, monitorización y gestión de incidencias.

El sistema se ha desplegado sin entorno gráfico, priorizando el rendimiento y la administración remota del servidor. Para su desarrollo se han utilizado tecnologías como Apache2, Perl CGI, MariaDB, Postfix, Dovecot, Grafana, Prometheus y Docker.

Además, se implementaron distintas medidas de automatización y seguridad, como activación de usuarios mediante correo electrónico, copias de seguridad automáticas, control de cuotas, monitorización del sistema y registro de accesos. Como funcionalidad adicional, se integró un sistema de inteligencia artificial basado en Ollama y Llama 3.2 para clasificar incidencias municipales automáticamente.

2. Configuración inicial del servidor

En primer lugar, se crearon los usuarios administrativos encargados de gestionar el sistema. A estos usuarios se les añadieron permisos sudo para poder realizar tareas de administración de forma segura sin utilizar directamente la cuenta root.

```
root@vnic-ecosalmantica:~# adduser sergio
root@vnic-ecosalmantica:~# adduser carlos
root@vnic-ecosalmantica:~# usermod -aG sudo sergio
root@vnic-ecosalmantica:~# usermod -aG sudo carlos
```

Posteriormente, se creó el grupo admins_eco, utilizado para agrupar a los administradores del sistema, así como los grupos ciudadanos y técnicos, que permiten diferenciar los permisos y servicios disponibles para cada tipo de usuario dentro de la plataforma.

```
root@vnic-ecosalmantica:~# groupadd admins_eco
root@vnic-ecosalmantica:~# usermod -aG admins_eco sergio
root@vnic-ecosalmantica:~# usermod -aG admins_eco carlos
```

```
carlos@vnic-ecosalmantica:~$ sudo groupadd ciudadanos
carlos@vnic-ecosalmantica:~$ sudo groupadd tecnicos

carlos@vnic-ecosalmantica:~$ cat /etc/group | grep -E "tecnicos|ciudadanos"
ciudadanos:x:1004:
tecnicos:x:1005:
```

Tras la configuración inicial de usuarios y grupos, se comenzó la preparación de los distintos servicios que formarían parte de la plataforma EcoSalmántica.

3. Configuración de Apache y plataforma web

3.1. Configuración inicial de Apache y HTTPS

Una vez configurados los usuarios y grupos del sistema, se creó el directorio /manuales_smartcity en la raíz del servidor para almacenar la documentación técnica utilizada por los operarios municipales. Este directorio se configuró con el grupo propietario tecnicos, permitiendo que únicamente los usuarios pertenecientes a dicho grupo pudieran trabajar sobre su contenido mediante los permisos correspondientes.

```
sergio@vnic-ecosalmantica:/$ sudo mkdir /manuales_smartcity
[sudo] password for sergio:
sergio@vnic-ecosalmantica:/$ ls
bin  dev  home  lost+found  media  opt  root  sbin  srv  tmp  var
boot  etc  lib  manuales_smartcity  mnt  proc  run  snap  sys  usr
sergio@vnic-ecosalmantica:/$ |

sergio@vnic-ecosalmantica:/$ sudo chown root:tecnicos /manuales_smartcity
sergio@vnic-ecosalmantica:/$ sudo chmod 2775 /manuales_smartcity
```

Posteriormente, se instaló y configuró Apache2 como servidor web principal de la plataforma. Además de habilitar su arranque automático mediante systemd, se configuró un acceso web al directorio de manuales mediante /var/www/html/manuales, permitiendo acceder desde el navegador al contenido almacenado en /manuales_smartcity. Para permitir conexiones externas también se configuró UFW habilitando los puertos necesarios para HTTP y SSH.

```
sergio@vnic-ecosalmantica:/$ sudo apt install apache2
```

```
sergio@vnic-ecosalmantica:/$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service scri
.
Executing: /lib/systemd/systemd-sysv-install enable apache2
```

```
sergio@vnic-ecosalmantica:/$ sudo apt install ufw
```

```
sergio@vnic-ecosalmantica:/$ sudo ufw status
Status: active

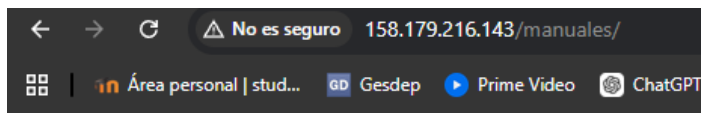
To Action From
--
22/tcp ALLOW Anywhere
80/tcp ALLOW Anywhere
22/tcp (v6) ALLOW Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)
```

Vemos la web inicial por el puerto 80:



Además añadimos un archivo de prueba a la carpeta `/manuales_smartcity` para ver si es accesible desde la web:

```
sergio@vnic-ecosalmantica:/var/www/html$ ls /manuales_smartcity/
test.txt
```



Index of /manuales

Name	Last modified	Size	Description
 Parent Directory	-		
 test.txt	2026-05-09 14:50	28	

Apache/2.4.52 (Ubuntu) Server at 158.179.216.143 Port 80

También, se habilitó el módulo userdir de Apache para proporcionar webs personales a los ciudadanos mediante directorios public_html dentro de sus cuentas de usuario. Como prueba de funcionamiento, se creó una web pública para el usuario Juan. También se configuró /etc/skel/public_html para que todos los nuevos usuarios creados en el sistema dispusieran automáticamente de una estructura básica para su web personal.

```
sergio@vnic-ecosalmantica:/var/www/html$ sudo a2enmod userdir
Enabling module userdir.
To activate the new configuration, you need to run:
systemctl restart apache2
```

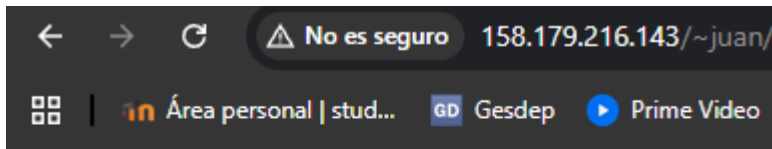
Aplicamos skel:

```
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo mkdir /etc/skel/public_html
```

Estructura de web para todos los usuarios:

```
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo cat /etc/skel/public_html/index.html
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Web Personal de {{USUARIO}}</title></head>
<body>
<h1>Web personal de {{USUARIO}}</h1>
<p>Ciudadano de EcoSalmántica Smart City</p>
</body>
</html>
```

```
juan@vnic-ecosalmantica:~$ ls
public_html
juan@vnic-ecosalmantica:~$ cd public_html/
juan@vnic-ecosalmantica:~/public_html$ ls
index.html
```



Web personal de Juan

Ciudadano registrado en Ecosalmántica.

Como última parte de la configuración inicial de la web se instaló y configuró Certbot para habilitar conexiones seguras HTTPS mediante certificados SSL/TLS asociados al dominio de la plataforma, garantizando así una comunicación cifrada entre los usuarios y el servidor.

Para ello registramos nuestra ip pública en <https://www.duckdns.org/>, con el dominio [ecosalmantica1.duckdns.org](https://www.duckdns.org/), una vez tenemos el dominio abrimos el puerto 443 tanto en UFW como en OracleCloud e instalamos Certbot y lo activamos junto a Apache2. Gracias a esta configuración, toda la plataforma pasó a funcionar mediante HTTPS, aumentando la seguridad de las conexiones y protegiendo las credenciales de los usuarios durante la autenticación.

```
sergio@vnic-ecosalmantica:/var/www/html$ sudo ufw allow 443/tcp
[sudo] password for sergio:
Rule added
Rule added (v6)
sergio@vnic-ecosalmantica:/var/www/html$ sudo snap install --classic certbot
certbot 5.5.0 from Certbot Project (certbot-eff✓) installed
sergio@vnic-ecosalmantica:/var/www/html$ sudo ln -s /snap/bin/certbot /usr/local/bin/certbot
sergio@vnic-ecosalmantica:/var/www/html$ sudo certbot --apache
```

3.2. Desarrollo CGI, autenticación y gestión web

3.2.1. Activación de CGI y gestión de sesiones

Tras finalizar la configuración inicial de Apache, se habilitó el soporte CGI para permitir la ejecución de scripts dinámicos desarrollados en Perl. A partir de esta configuración se desarrolló el sistema principal de autenticación y gestión web de la plataforma.

```
sergio@vnic-ecosalmantica:/var/www/html$ sudo a2enmod cgi
```

Para mantener las sesiones activas entre páginas, se creó un directorio privado de almacenamiento de sesiones accesible únicamente por el servidor web. Gracias a ello, los usuarios pueden mantenerse autenticados durante toda su navegación dentro de la plataforma sin necesidad de volver a iniciar sesión continuamente.

```
carlos@vnic-ecosalmantica:/var/www/html/portal$ sudo mkdir /var/lib/cgi-sessions
carlos@vnic-ecosalmantica:/var/www/html/portal$ sudo chown www-data:www-data /var/lib/cgi-sessions
carlos@vnic-ecosalmantica:/var/www/html/portal$ sudo chmod 700 /var/lib/cgi-sessions
```

3.2.2. Autenticación mediante pwauth

Para permitir que los usuarios ya existentes en el sistema operativo puedan entrar y autenticarse sin poner en peligro la seguridad de los archivos de contraseñas, se instaló la utilidad pwauth

```
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo apt install pwauth
Reading package lists... Done
```

```
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ cat /etc/apache2/conf-available/pwauth.conf
SetEnv PWAUTH_WITHW 1
```

```
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo a2enconf pwauth
Enabling conf pwauth.
To activate the new configuration, you need to run:
    systemctl reload apache2
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo systemctl restart apache2
```

Es un binario que permite que www-data compruebe las credenciales de cada usuario son válidas consultado directamente a PAM, pero sin tener permisos de lectura sobre el archivo shadow

Para su correcto funcionamiento en un entorno CGI, se creó un archivo de configuración en Apache habilitando la variable de entorno PWAUTH_WITHW. Esto permite que el binario acepte las peticiones del servidor web.

Implementar este binario garantiza la seguridad del sistema debido a los siguientes factores:

- Privilegios controlados (Bit SUID): Por defecto el archivo /etc/shadow sólo es accesible por el root. Si el servidor web trabaja bajo el usuario sin privilegios www-data no tiene permisos para leer dicho archivo. Para poder hacer login de forma segura el binario pwauth tiene el bit SUID activo, el programa se ejecuta temporalmente con los privilegios de su propietario (root), realice la comprobación y libere los privilegios al finalizar.

- Delegación de autenticación en el sistema PAM: El binario pwauth no realiza el mismo la verificación de las credenciales por sí mismo ni lee el archivo de contraseñas directamente, en su lugar delega la tarea al sistema PAM del sistema operativo, invoca la configuración específico en /etc/pam.d/pwauth

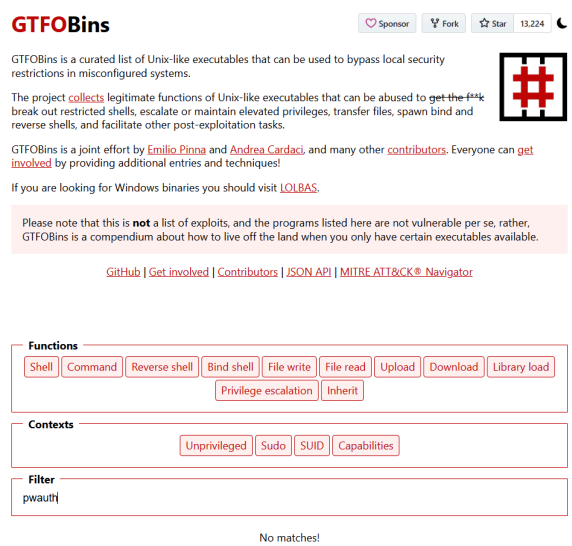
```
ls -l /etc/pam.d/
```

```
-rw-r--r-- 1 root root 315 Aug 21 2021 pwauth
```

Esto tiene ventajas de seguridad en el proyecto:

- Abstracción del almacenamiento de credenciales: El entorno web no necesita saber ni cómo se almacena el sistema operativo los hashes de las contraseñas. Si en el futuro el servidor Linux cambia su algoritmo de cifrado el sistema de autenticación web seguirá funcionando sin modificar nada.
- Heredabilidad de directivas de seguridad: Al centralizar la validación en PAM cualquier política global del servidor (como el bloqueo automático de cuentas tras un número determinado de fallos) es adoptada inmediatamente por el formulario web, protegiendo el sistema. Por ejemplo añadimos que con 3 fallos se bloqueará la cuenta 30 segundos.
- Aislamiento del archivo crítico: Se evita por completo que www-data tenga acceso directo de lectura sobre /etc/shadow. Esto quita el riesgo de que por ejemplo en una vulnerabilidad en la aplicación web, como por ejemplo LFI acabe con una filtración de las contraseñas del sistema.

Una librería muy importante para login.pl es IPC::Open2, permite abrir el canal de comunicación con procesos externos (como pwauth). Su uso es obligatorio para pasar la información por la entrada estándar (STDIN), evitando que aparezca en el listado de procesos del sistema.



(Buscando en GTFOBins no encontramos ninguna coincidencia de una escalada de privilegios mediante los archivos SUID identificados)

3.2.3. Registro y activación de usuarios

Para explicar los diferentes binarios que se utilizan para registrar y dar de alta en el sistema a nuevos usuarios utilizaré un flujo de ejecución que separa las peticiones web de las tareas de kernel

Tras la toma de contacto inicial, el ciudadano es dirigido a login.html. Ignoraremos el login y pulsaremos sobre la opción de registro, comenzando así el proceso.

En esta etapa el proceso va al formulario de inscripción ubicado en /var/www/registro/index.html. Al rellenarse los datos se procesan por el script registro.pl, que se encarga de su almacenamiento en la base de datos.

Es importante destacar que el usuario se mantiene en la base de datos en estado 'pendiente' y no tiene aún una entrada en /etc/shadow. Esto permite que el sistema operativo no se altere hasta que el usuario verifique el correo.

usuario_id	nombre	email	estado	token	expira_token
44	luciano	luchamosporlavidatiktok@gmail.com	pendiente	11110	2026-05-16 09:15:04

Después de registro.pl el flujo llega a confirmación.pl, el sistema espera que el ciudadano meta el token único que se ha enviado previamente. Una vez recibido el código el script realiza una validación con la base de datos para asegurar validez y que no esté caducado. Si la comprobación es exitosa se actualizará el estado del usuario a 'confirmado'. Hasta aquí llega la interacción con la web y activa el disparador para que los procesos

Con el usuario en estado confirmado llega el turno del script /root/scripts/activación_usuarios.pl. El proceso manejado mediante una tarea Cron con ejecución cada minuto actúa como la base de administración de alta en el sistema.

```
my $sth = $dbh->prepare("
    SELECT id, login, rol, pass_tmp
    FROM usuarios
    WHERE estado = 'confirmado'
");
$sth->execute();
```

```
$dbh->do(
    "UPDATE usuarios SET estado='activo', pass_tmp=NULL WHERE id=?",
    undef,
    $u->{id}
);
```

En esta fase el script activa la cuenta en la base de datos e inicia las cuotas utilizando la librería Quota, asignando límites para cada rol: 40 MB para el almacenamiento de informes de ciudadanos y 100 MB para documentación y manuales de los técnicos.

```
sub aplicar_cuota {
    my ($login, $uid, $rol) = @_;

    my ($soft, $hard) = ($rol eq 'tecnico')
        ? (102400, 122880)
        : (40960, 51200);

    my $dev = Quota::getqcarg('/');

    unless ($dev) {
        print "[ERROR CUOTA] No se pudo obtener dispositivo de cuota para /\n";
        return;
    }

    my $res = Quota::setqlim($dev, $uid, $soft, $hard, 0, 0);

    if (defined $res && $res == 0) {
        print "[CUOTA] Cuota aplicada a $login: soft=$soft KB hard=$hard KB\n";
    } else {
        my $err = Quota::strerr() || $! || "error desconocido";
        print "[ERROR CUOTA] No se pudo aplicar cuota a $login: res="
            . (defined $res ? $res : "undef")
            . " error=$err dev=$dev uid=$uid\n";
    }
}
```

Para añadir los nuevos usuarios al sistema usamos librerías como Linux::usermod, que permite la gestión de /etc/passwd sin invocar comandos externos. Es una capa de abstracción que manipula los registros de forma atómica y segura. Además Passwd::Unix, que permite gestionar el archivo /etc/shadow. Se utiliza para meter de forma segura el hash de la contraseña del ciudadano.

Tras la ejecución de este script el usuario 'luciano' queda correctamente registrado tanto en la base de datos como activo y en el /etc/passwd y /etc/shadow

```
luciano:x:1010:1004:/:/home/luciano:/bin/bash
```

(etc/passwd)

```
luciano:$y$j9T$XwpFnXh5Mfy82y9q$A4IIvi7o8mxtT7bUEZZxzKu.qzFuPmQjnc0ZuClpve2:20588:0:99999:7:::
```

(etc/shadow)

luciano	luchamosporlavidatiktok@gmail.com	activo	2026-05-15 09:19:43
---------	-----------------------------------	--------	---------------------

(base de datos, tabla usuario)

El flujo se podría ver así:



(imagen generada por chat gpt)

3.2.4. Gestión de documentación técnica

Además, se desarrolló el script principal encargado de gestionar los manuales técnicos desde la interfaz web. Este CGI permite a los técnicos listar, subir, modificar y borrar los archivos del directorio /manuales_smartcity directamente desde la web.

```
sergio@vnic-ecosalmantica:~$ sudo nano /manuales_smartcity/index.pl
```

Antes de realizar cualquier operación, el sistema carga la sesión activa del usuario para comprobar si pertenece al grupo técnicos, permitiendo así el acceso únicamente a los operarios autorizados.

```
my $session = CGI::Session->load(
    undef,
    $cgi,
    { Directory => '/var/lib/cgi-sessions' }
);

my $usuario = $session->param('usuario');

my $es_tecnico = $session->param('es_tecnico');
```

Para una mayor seguridad, se han implementado comprobaciones sobre nombres, extensiones y tipos de archivos subidos permitiendo solo .txt verdaderos. La primera comprobación evita accesos a directorios externos mediante rutas manipuladas y archivos peligrosos con doble extensión como podría ser un ataque.php.txt.

```
return 0 if $nombre =~ m{/};

return 0 if $nombre =~ m{\\.\\.};

return 0 unless $nombre =~ /^[a-zA-Z0-9_-]+\\.txt$/;
```

Además, mediante el uso de MIME se comprueba desde el navegador que el archivo subido se corresponde realmente a texto plano.

```
my $mime = $cgi->uploadInfo($filehandle)->{'Content-Type'};
```

Cada acción realizada por los usuarios queda registrada automáticamente en un archivo log específico, permitiendo mantener un control detallado sobre la actividad del sistema. También se evitó el reenvío accidental de formularios al actualizar la página tras realizar operaciones.

```
escribir_log("MODIFICADO $archivo");
```

```
sub escribir_log {  
    my ($accion) = @_;  
    open(my $log, ">>", $LOG);  
    my $fecha = localtime();  
    print $log "[$fecha] $usuario - $accion\n";  
    close($log);  
}
```

Después de completar acciones como subir, modificar o eliminar archivos, el CGI realiza una redirección automática a la página principal de manuales en lugar de mostrar directamente el resultado del formulario. De esta manera se evita que el navegador vuelva a enviar la misma petición al pulsar F5 o recargar la página, impidiendo duplicados involuntarios de operaciones.

```
print $cgi->redirect('/manuales/');
```

A nivel de permisos, el directorio de manuales y los archivos log se configuraron utilizando www-data como propietario y tecnicos como grupo, limitando el acceso únicamente al propietario y al grupo autorizado. Finalmente, se establecieron los permisos correspondientes sobre el CGI y se reinició Apache2 para aplicar toda la configuración.

```
sergio@vnic-ecosalmantica:~$ sudo chown -R www-data:tecnicos /manuales_smartcity
```

Establece permisos completos de lectura, escritura y ejecución para el propietario y el grupo mientras que el resto de usuarios del sistema no tendrá ningún permiso sobre el directorio.

```
sergio@vnic-ecosalmantica:~$ sudo chmod 770 /manuales_smartcity
```

Permite únicamente al propietario del script CGI leerlo, modificarlo y ejecutarlo

```
sergio@vnic-ecosalmantica:~$ sudo chmod 700 /manuales_smartcity/index.pl
```

```
sergio@vnic-ecosalmantica:~$ sudo touch /var/log/manuales.log
```

Asigna como propietario del archivo log al usuario www-data, permitiendo que Apache pueda escribir directamente en él durante la ejecución de los CGI.

```
sergio@vnic-ecosalmantica:~$ sudo chown www-data:adm /var/log/manuales.log
sergio@vnic-ecosalmantica:~$ sudo chmod 640 /var/log/manuales.log
sergio@vnic-ecosalmantica:~$ sudo systemctl restart apache2
```

Finalmente, index.pl fue movido al directorio cgi-bin y dentro del directorio web /manuales se creó un index.html encargado de redirigir automáticamente al CGI principal de gestión documental.

```
sergio@vnic-ecosalmantica:~$ sudo mv /manuales_smartcity/index.pl /usr/lib/cgi-bin/manuales.pl
sergio@vnic-ecosalmantica:~$ sudo chown www-data:www-data /usr/lib/cgi-bin/manuales.pl
sergio@vnic-ecosalmantica:~$ sudo chmod 700 /usr/lib/cgi-bin/manuales.pl
sergio@vnic-ecosalmantica:~$ sudo nano /var/www/html/manuales/index.html
```

3.2.5 Acceso remoto mediante SFTP

Además de la plataforma web, también se habilitó el acceso remoto mediante SFTP para permitir a los usuarios gestionar archivos directamente desde sus directorios personales del servidor.

SFTP funciona sobre el propio servicio SSH del sistema, por lo que no fue necesario instalar un servidor FTP adicional. Gracias a ello, las transferencias de archivos se realizan de forma cifrada y segura utilizando las mismas credenciales y permisos configurados previamente para cada usuario Linux dentro de EcoSalmántica, permitiendo acceder remotamente a documentos, webs personales y archivos almacenados en el servidor.

```
PS C:\Users\SERGI> sftp sergio@ecosalmantica1.duckdns.org
(sergio@ecosalmantica1.duckdns.org) Password:
Connected to ecosalmantica1.duckdns.org.
sftp> ls
informe_diario.pl  mail                prueba.pl           prueba.sql          public_html
sftp>
```


3.2.6 Recuperación de la contraseña

En el caso de que cualquier usuario registrado en el sistema haya olvidado su contraseña tendrá la posibilidad de recuperarla a través del login principal. El proceso comienza al pulsar en “ha olvidado su contraseña”, tras ello se le dirigirá hacia `recuperacion/index.html`, un formulario html que tendrá que rellenar con su login y correo electrónico.



Una vez rellenando y solicitada la recuperación el script `recuperar.pl` realiza las siguientes operaciones:

```
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo cat recuperacion.pl
```

```
my $sth = $dbh->prepare(
    "SELECT id, nombre, email
    FROM usuarios
    WHERE login = ?
    AND email = ?
    AND estado = 'activo'
    LIMIT 1"
);
```

- Validará que en la tabla usuarios esté el login y el email, además de que la cuenta esté en activo

```
# generar código
my $codigo = 10000 + int(rand(90000));

# borrar anteriores
$dbh->do(
    "DELETE FROM clave_recuperacion
    WHERE usuario_id = ?",
    undef,
    $u->{id}
);

# guardar código
$dbh->do(
    "INSERT INTO clave_recuperacion
    (usuario_id, clave, expira)
    VALUES (?, ?, DATE_ADD(NOW(), INTERVAL 15 MINUTE))",
    undef,
    $u->{id},
    $codigo
);
```

- Se generará un código aleatorio de 5 dígitos con una validez de 15 minutos

```
eval {
    my $msg = MIME::Lite->new(
        From => 'carloscuadrado0606@gmail.com',
        To   => $u->{email},
        Subject => 'Recuperación EcoSalmantica',
        Data  => <<"MAIL"
    Hola $u->{nombre},

    Tu código de recuperación es:

    $codigo
    >>
    );
```

- Y por último se enviará un correo electrónico con el token si todo ha salido de manera correcta

Una vez que el usuario recibe el código, accede a un formulario donde introduce el token y su nueva contraseña. El script valida con una expresión regular que la contraseña cumpla la política.

Para que la contraseña sea compatible con Linux se genera un Salt aleatorio y se aplica el algoritmo de hashing (simulado con \$y\$j9T, estándar en sistemas modernos como Debian 12).

En lugar de modificar el sistema diferente, el hash se inserta en cambios_password

```
$dbh->do(
    "INSERT INTO cambios_password (login, hash_nuevo)
    VALUES (?, ?)",
    undef,
    $r->{login},
    $hash
);
```

Por último el script /root/scripts/cambios_password.pl ejecutado por el Cron con privilegios de root finaliza este nuevo flujo:

- Aislamiento: El servidor no toca el archivo /etc/shadow. Solo lo escribe en una tabla de MariaDB

- Utilizamos de nuevo Passwd::Unix para abrir y modificar de forma segura el registro del usuario en el sistema
- Validación de seguridad: Antes de aplicar el cambio el script de root vuelve a verificar que el usuario sigue activo en la base de datos y que el hash tiene el formato correcto, esto podría ayuda a enviar inyecciones de código en el sistema
- Una vez actualizada la contraseña en el sistema operativo, el registro se elimina de la tabla cambios_password

Para la ejecución de este Cron el script tendrá que encontrar algún usuario activo con tabla cambios_password, este mismo script es el que utilizaremos para hacer cambios en el perfil del usuario y explicaremos a continuación.

3.2.7 Cambios en el perfil del usuario

Para que cada perfil sea libre de cambiar sus datos personales se creó el panel de control perfil.pl. protegido mediante control de sesiones (CGI::Session). El módulo actúa como el centro donde el usuario puede gestionar su ciclo de vida, abarca tres operaciones muy importantes dentro del sistema: actualizar datos (no de tanta importancia) y cambiar la contraseña y baja del servicio.

```
carlos@vnica-ecosalmantica:/usr/lib/cgi-bin$ sudo cat perfil.pl
#!/usr/bin/perl
use strict;
use warnings;
use utf8;
use CGI;
use CGI::Session;
use DBI;
binmode(STDOUT, ":encoding(UTF-8)");

my $cgi      = CGI->new;
my $session = CGI::Session->new("driver:File", $cgi, {Directory => '/var/lib/cgi-sessions'});
```

3.2.7.1 Modificación de datos personales, cambiar_datos.pl

Este script permite actualizar el nombre y el correo electrónico del usuario. Para realizar cualquier cambio el usuario debe introducir la contraseña actual y el script invoca a pwauth para validar sus credenciales.

Se utilizan expresiones regulares para validar correo y nombre. Además se realiza una consulta a la base de datos para evitar que el correo no pertenezca a otro usuario.

```
carlos@vnica-ecosalmantica:/usr/lib/cgi-bin$ sudo nano cambiar_datos.pl
```

3.2.7.2 Actualizar contraseña, cambiar_password.pl

El cambio de contraseña se realiza de manera muy similar a la recuperación de contraseña, este script también necesita la contraseña actual y exige que la nueva contraseña cumpla los requisitos.

La nueva clave se cifra con el estándar \$y\$ y un Salt aleatorio de 16 caracteres. Como explicamos en el apartado anterior de recuperación de contraseñas no ejecutamos comandos privilegiados desde la web, el nuevo hash se inserta en la tabla cambios_password. Será el Cron mediante /root/scripts/cambios_password.pl quien aplique el cambio de contraseña en menos de un minuto.

```
my $salt = '$y$j9T$' . join('', map { ('a'..'z','A'..'Z','0'..'9')[rand 62] } 1..16);
my $hash = crypt($pass_nueva, $salt);

my $dbh = DBI->connect(
    "DBI:mysql:database=ecosalmantica;host=localhost",
    "admineco", "admineco123",
    { RaiseError => 1, mysql_enable_utf8 => 1 }
);

# Borrar cambio pendiente anterior si existe
$dbh->prepare("DELETE FROM cambios_password WHERE login = ?")
->execute($usuario);

# Encolar nuevo hash para que el cron lo aplique en shadow
$dbh->prepare("INSERT INTO cambios_password (login, hash_nuevo) VALUES (?, ?)")
->execute($usuario, $hash);

$dbh->disconnect;
redirigir("Contraseña actualizada. Se aplicará en menos de 1 minuto.", "ok");
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ |
```

3.2.7.3 Baja y destrucción de la sesión

Tras validar la identidad con la contraseña actual, el estado del usuario en la base de datos será 'baja', y se registrará el timestamp exacto. Este script no invoca comandos. Delega también la eliminación física del entorno (/home y cuotas) a los scripts de root ejecutados por el root.

Finalmente el sistema destruye el archivo de sesión en el servidor /var/lib/cgi-sessions y envía una cabecera HTTP para invalidar y rechazar la cookie en el navegador.

```
$dbh->disconnect;

# 5. CIERRE DE SESIÓN Y DESPEDIDA
# Borramos la sesión en el servidor
$session->delete();
$session->flush();

# Borramos la cookie en el navegador
my $cookie = $cgi->cookie(
    -name => 'CGISESSID',
    -value => '',
    -expires => '-1d',
    -path => '/'
);

# Redirigimos a la página de inicio de la ciudad (raiz)
print $cgi->redirect(
    -uri => '/',
    -cookie => $cookie
);

exit;
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ |
```

3.2.8 Mantenimiento y recolección de basura

En este entorno la base de datos puede llenarse rápidamente de solicitudes inconclusas o tokens caducados. Para garantizar limpieza y eficiencia se ha implementado un sistema automatizado de higiene.

En este caso hemos optado por crear un nuevo usuario del sistema que únicamente realice esta tarea:

```
root@vnic-ecosalmantica:~/scripts# sudo adduser --system --group script_limpieza
Adding system user `script_limpieza' (UID 117) ...
Adding new group `script_limpieza' (GID 125) ...
Adding new user `script_limpieza' (UID 117) with group `script_limpieza' ...
Creating home directory `/home/script_limpieza' ...
```

Al usar `--system` el usuario no tiene shell de acceso ni carpet home, lo que impide que nadie pueda loguearse con él por SSH o consola. Además el script de limpieza no corre con privilegios de root, sino bajo esta identidad limitada. Si el script fuese secuestrado el atacante no tendría permisos administrativos.

```
root@vnic-ecosalmantica:/opt/ecosalmantica/scripts# chown script_limpieza:script_limpieza /opt/ecosalmantica/scripts/limpieza_eco.pl
```

```
root@vnic-ecosalmantica:/opt/ecosalmantica/scripts# chmod 700 /opt/ecosalmantica/scripts/limpieza_eco.pl
```

Se le transfiere la propiedad del script al usuario creado y con `chmod 700` el dueño, el mismo, puede leer, escribir o ejecutar el archivo. Este protege las credenciales de la base de datos que están en el código, evitando que cualquier otro usuario pueda ver la contraseña.

Además se creó un nuevo usuario de la base de datos llamado `cron_limpieza` que no tiene acceso total. Se le han dado permisos muy controlados:

```
MariaDB [ecosalmantica]> GRANT SELECT, DELETE ON ecosalmantica.usuarios TO 'cron_limpieza'@'localhost';
Query OK, 0 rows affected (0.002 sec)

MariaDB [ecosalmantica]> GRANT DELETE ON ecosalmantica.tokens_verificacion TO 'cron_limpieza'@'localhost';
Query OK, 0 rows affected (0.001 sec)

MariaDB [ecosalmantica]> GRANT DELETE ON ecosalmantica.clave_recuperacion TO 'cron_limpieza'@'localhost';
Query OK, 0 rows affected (0.001 sec)
```

GRANT SELECT, DELETE: El script solo puede leer para identificar caducidades y

borrar registros antiguos. No puede insertar datos falsos ni modificar perfiles existentes, limitando la exposición.

Además los permisos se limitan solamente a las tablas de usuarios, tokens_verificacion y clave_recuperacion, protegiendo el resto de la infraestructura del sistema.

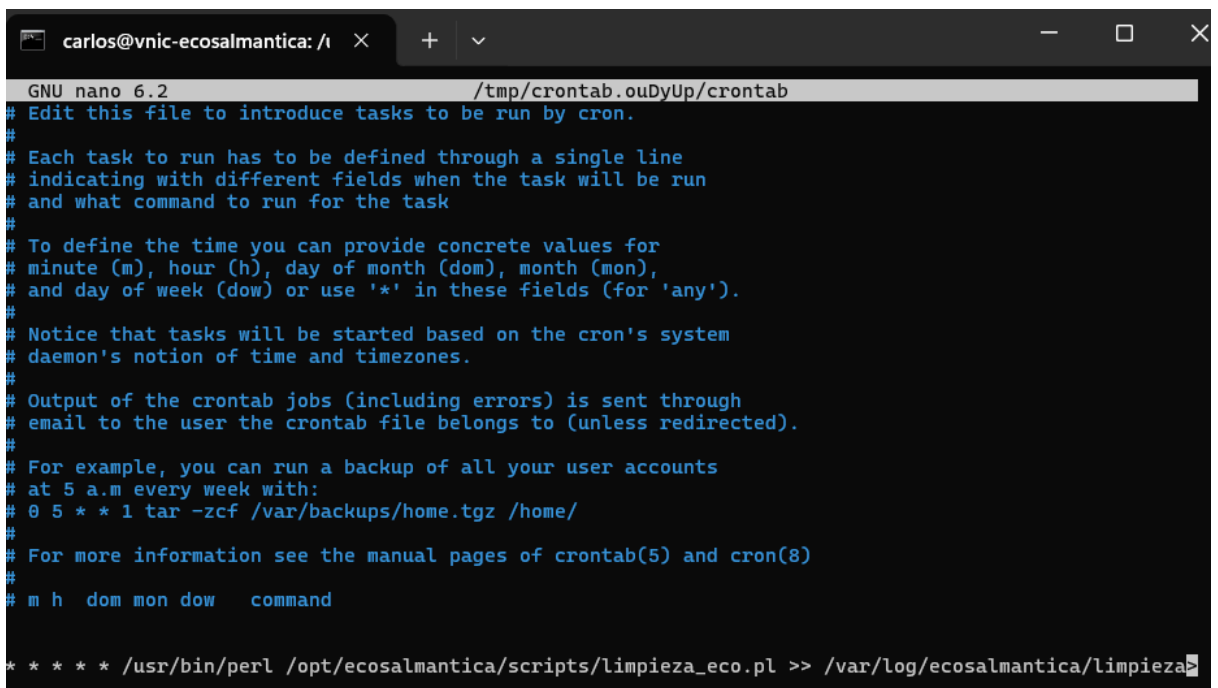
Finalmente se ha habilitado un sistema de registro de actividad en /var/log/limpieza_eco.log:

```
root@vnic-ecosalmantica:/opt/ecosalmantica/scripts# touch /var/log/limpieza_eco.log
root@vnic-ecosalmantica:/opt/ecosalmantica/scripts# chown script_limpieza:script_limpieza /var/log/limpieza_eco.log
```

Se crea el archivo y se otorga la propiedad al usuario del script para que pueda escribir en él sin hacer sudo. Lo que permite al administrador supervisar que la recolección de basura se ejecute correctamente y escribe cuántos registros caducados se eliminan en cada ciclo del Cron.

Por último paso lo añadiremos en el Cron:

```
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo crontab -u script_limpieza -e
```



```
GNU nano 6.2 /tmp/crontab.ouDyUp/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
* * * * * /usr/bin/perl /opt/ecosalmantica/scripts/limpieza_eco.pl >> /var/log/ecosalmantica/limpieza
```

3.2.9 Dashboard dinámico

Para poder ver el estado de los servicios del sistema se utiliza una técnica de sondeo activo por demanda, cada vez que un usuario carga la página, tanto inicio.pl como portal.pl, el servidor realiza una comprobación instantánea del estado de los servicios.

Para verificar si un servicio está fusionando ambos scripts utilizan el módulo IO::Socket::INET. Esta librería permite al script actuar como un cliente de red básico que intenta saludar a los servicios locales:

```
GNU nano 6.2 inicio.pl
#!/usr/bin/perl
use strict;
use warnings;
use utf8;
use CGI;
use IO::Socket::INET;

# Configuración para que el navegador entienda tildes y ñ
binmode(STDOUT, ":encoding(UTF-8)");
my $cgi = CGI->new;

sub check {
    my ($port) = @_;
    my $socket = IO::Socket::INET->new(
        PeerAddr => 'localhost',
        PeerPort => $port,
        Proto => 'tcp',
        Timeout => 1
    );
    return $socket ? '<b style="color:green">OPERATIVO</b>' : '<b style="color:red">CAÍDO</b>';
}

# Realizamos los chequeos
my $s_web = check(80);
my $s_ssl = check(443);
my $s_ssh = check(22);
my $s_sql = check(3306);
```

```
GNU nano 6.2 portal.pl
#!/usr/bin/perl
use strict;
use warnings;
use utf8;
use CGI;
use CGI::Session;
use IO::Socket::INET;

# Configuración para tildes
binmode(STDOUT, ":encoding(UTF-8)");

my $cgi = CGI->new;
my $session = CGI::Session->new("driver:File", $cgi, {Directory => '/var/lib/cgi-sessions'});

# Verificación de sesión
unless ($session && $session->param('usuario')) {
    print $cgi->redirect('/login/');
    exit;
}

my $usuario = $session->param('usuario');
my $es_tecnico = $session->param('es_tecnico') || 0;

# Monitoreo
sub check_service {
    my ($host, $port) = @_;
    my $socket = IO::Socket::INET->new(
        PeerAddr => $host,
        PeerPort => $port,
        Proto => 'tcp',
        Timeout => 1
    );
    if ($socket) {
        close($socket);
        return '<b style="color:white; background:#008000; padding:4px 10px; border-radius:4px; font-size:12px;">OPERATIVO';
    } else {
        return '<b style="color:white; background:#cc0000; padding:4px 10px; border-radius:4px; font-size:12px;">CAÍDO';
    }
}

my $s_web = check_service('localhost', 80);
my $s_ssl = check_service('localhost', 443);
my $s_ssh = check_service('localhost', 22);
my $s_sql = check_service('localhost', 3306);
```

El script intenta abrir un socketTCP hacia una dirección localhost y un puerto específico. Configurando un Timeout de 1, si el servicio no responde en un segundo el script asume que está caído, evitando que la carga de la web se bloquee si un servicio falla.

Se monitorea ApacheWeb Apache,80; SSH,22; SSL,443 y MariaDB, 3306

Estado Servicios (Tiempo Real)

- Apache Web: **OPERATIVO**
- HTTPS SSL: **OPERATIVO**
- SSH/SFTP: **OPERATIVO**
- Base de Datos: **OPERATIVO**

3.2.10 Cron

Para terminar se explica el Cron que actúa en segundo plano constantemente para ejecutar las tareas administrativas más críticas del sistema, que no pueden ser delegadas al servidor web por restricciones de seguridad.

Vamos a repasar algunos de los binarios que hemos nombrado anteriormente:

- cambios_password.pl, el puente entre la base de datos y el archivo /etc/shadow, lee los nuevos hashes y los aplica al sistema mediante la librería Passwd::Unix
- baja_usuarios.pl, ejecuta la eliminación de las cuentas, al detectar usuarios con estado 'baja', procede a eliminar sus procesos activos, borrar su directorio /home, limpiar cuotas de disco y suprimir el usuario de manera definitiva.
- limpieza_eco.pl, elimina los tokens de recuperación y elimina los registros de usuarios que no confirmaron su registro en un plazo de 5 minutos, evitando que la base de datos se sature.
- activacion_usuarios.pl, materializa el alta de nuevos ciudadanos, detecta usuarios recién verificados en la web y se encarga de crear su entorno de sistema, creación del UID, asignación de la shell, creación del directorio personal y configuración de las cuotas de disco iniciales

Se encuentran en dos lugares diferentes, porque son de usuarios diferentes, para ver el de limpieza, un script de un usuario de sistema:


```
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo nano perlcat.pl
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo crontab -u script_limpieza -e|
```

```
GNU nano 6.2 /tmp/crontab.NE9JWA/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
* * * * * /usr/bin/perl /opt/ecosalmantica/scripts/limpieza_eco.pl >> /var/log/ecosalmantica/limpieza
```

y para ver los de root:

```
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo crontab -e|
```

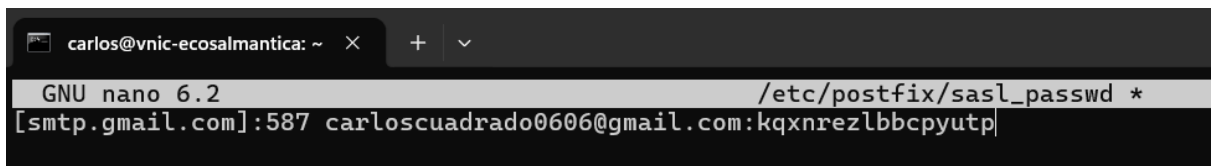
```
GNU nano 6.2 /tmp/crontab.BLcl78/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
* * * * * /usr/bin/perl /root/scripts/gestion_usuarios.pl >> /var/log/gestion_usuarios.log 2>&1
0 3 * * * /usr/bin/perl /root/backup_sistema.pl >> /var/log/backups.log 2>&1
* * * * * /usr/bin/perl /root/scripts/activacion_usuarios.pl >> /var/log/ecosalmantica/activaciones.log
* * * * * /usr/bin/perl /root/scripts/cambios_password.pl >> /var/log/ecosalmantica/passwords.log 2>>&1
* * * * * /usr/bin/perl /root/scripts/bajas_usuario.pl >> /var/log/ecosalmantica/bajas.log 2>> /var/l
```

Los * * * * * significa que se lanzará cada minuto, hemos optado por elegir que se lance cada minuto para poder hacer comprobaciones rápidas del sistema, en un entorno más seguro se lanzarían cada más tiempo, sobretodo algunas como la de limpieza, con la finalidad de disminuir la posibilidad de condiciones de carreras y demás.

3.2.11 Configuración notificaciones externas

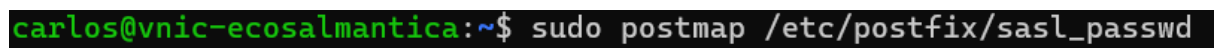
Para poder enviar a través del correo electrónico tokens de verificación o códigos de recuperación se configura Postfix para que actúe como SMTP a través de los servidores de Gmail.

La autenticación con el servidor externo requiere almacenar credenciales de forma que Postfix pueda procesarlas. Por eso en primer lugar editamos el archivo `/etc/postfix/sasl_passwd`, mapeando el servidor de relevo con la cuenta de gmail que enviará el correo y la clave de aplicación dedicada de Google



```
carlos@vnic-ecosalmantica: ~  
GNU nano 6.2 /etc/postfix/sasl_passwd *  
[smtp.gmail.com]:587 carloscuadrado0606@gmail.com:kqxnrezlbbcpyutp
```

Tras guardar el archivo se ejecuta:



```
carlos@vnic-ecosalmantica:~$ sudo postmap /etc/postfix/sasl_passwd
```

Compila el archivo de texto en una base de datos indexada: `sasl_passwd.db` que Postfix lee.

Para que Postfix procese de manera correcta el SMTP hay que añadir algunas directivas en el archivo de configuración de Postfix.

```
carlos@vnic-ecosalmantica: ~  
GNU nano 6.2 /etc/postfix/main.cf *  
mailbox_size_limit = 4194304  
message_size_limit = 3145728  
  
recipient_delimiter = +  
inet_interfaces = all  
inet_protocols = all  
  
smtpd_sasl_auth_enable = yes  
smtpd_sasl_type = dovecot  
smtpd_sasl_path = private/auth  
smtpd_sasl_security_options = noanonymous  
smtpd_sasl_authenticated_header = yes  
maillog_file = /var/log/postfix.log  
  
relayhost = [smtp.gmail.com]:587  
  
smtp_sasl_auth_enable = yes  
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd  
smtp_sasl_security_options = noanonymous  
  
smtp_use_tls = yes  
smtp_tls_security_level = encrypt  
smtp_tls_CAfile = /etc/ssl/certs/ca-certificates.crt  
  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo      M-G Copy
```

El parámetro relayhost redirige el tráfico de correo saliente hacia el servidor SMTP de Google en el puerto de entrega 587. Por otra parte con smtp_tls_CAfile se indica la ruta donde se almacenan las entidades de certificación del sistema, necesario para validar que el certificado es válido.

3.3. Blog ciudadano y participación web

Como parte de los servicios destinados a la participación ciudadana, se desarrolló un sistema de blogs accesible desde la plataforma web. Para ello, se creó el directorio /blogs_ciudadanos, encargado de almacenar las publicaciones realizadas por los usuarios.

```
sergio@vnic-ecosalmantica:~$ sudo mkdir /blogs_ciudadanos
```

Posteriormente, se configuraron los permisos del directorio utilizando www-data como propietario para permitir el acceso desde Apache y limitando el acceso únicamente a los usuarios autorizados.

```
sergio@vnic-ecosalmantica:~$ sudo chown www-data:www-data /blogs_ciudadanos  
sergio@vnic-ecosalmantica:~$ sudo chmod 770 /blogs_ciudadanos
```

El funcionamiento principal del blog se desarrolla mediante el CGI blog.pl. Las publicaciones se guardan como archivos .txt dentro del directorio /blogs_ciudadanos,

y el script se encarga de abrir automáticamente cada archivo, leer su contenido y extraer los distintos campos de la publicación, como el título, autor o contenido del mensaje. El script también comprueba si tenemos sesión iniciada para permitir o no escribir mensajes y dar likes en el foro.

```
sergio@vnic-ecosalmantica:~$ sudo nano /usr/lib/cgi-bin/blog.pl
```

Finalmente, se asignaron permisos 700 sobre blog.pl para permitir únicamente al propietario leer, modificar y ejecutar el CGI, evitando accesos no autorizados desde otras cuentas del sistema.

```
sergio@vnic-ecosalmantica:~$ sudo chown www-data:www-data /usr/lib/cgi-bin/blog.pl
sergio@vnic-ecosalmantica:~$ sudo chmod 700 /usr/lib/cgi-bin/blog.pl
```

En la siguiente imagen se puede observar parte del código encargado de crear nuevas publicaciones dentro del foro ciudadano, almacenando cada entrada en archivos .txt independientes junto a información como autor, fecha y contenido del mensaje.

```
my $id = scalar(@posts) + 1;
my $archivo = "$RUTA/post_$id.txt";
open(my $fh, ">:encoding(UTF-8)", $archivo);
print $fh "AUTOR:$usuario\n";
print $fh "FECHA:" . localtime() . "\n";
print $fh "LIKES:0\n";
print $fh "LIKEDBY:\n";
print $fh "CONTENIDO:\n";
print $fh "$mensaje\n";
close($fh);
}

print $cgi->redirect('/blog/');
```

Para facilitar el acceso, se creó dentro del directorio web un index.html que redirige automáticamente al CGI principal del blog. Finalmente, se reinició Apache2 para aplicar toda la configuración realizada.

```
sergio@vnic-ecosalmantica:~$ sudo mkdir /var/www/html/blog
```

```
sergio@vnic-ecosalmantica:~$ sudo nano /var/www/html/blog/index.html
```

4. Monitorización y estadísticas

4.1. Instalación de Docker, Grafana y Prometheus

Para la monitorización del sistema se desplegó una infraestructura basada en Prometheus, Grafana y Node Exporter utilizando contenedores Docker. Esta solución permite supervisar en tiempo real distintos parámetros del servidor, como el uso de CPU, memoria RAM y almacenamiento.

Se eligieron Grafana y Prometheus como tecnologías principales de monitorización debido a que nuestro seminario de la asignatura estuvo centrado en estas herramientas. Gracias a ello, ya contábamos con conocimientos previos sobre su funcionamiento, despliegue y configuración, lo que permitió integrarlas de forma más eficiente dentro del servidor y aprovechar sus capacidades.

En primer lugar instalamos Docker y Docker compose para facilitar el despliegue y control de los servicios de monitorización.

```
sergio@vnic-ecosalmantica:/monitorizacion$ sudo apt install docker.io
```

```
sergio@vnic-ecosalmantica:/monitorizacion$ sudo apt install docker-compose
```

Creamos también el directorio /monitorización para almacenar los archivos de configuración, dentro de este directorio se configuraron los archivos docker-compose.yml y prometheus.yml, definiendo los servicios, puertos y objetivos de monitorización del sistema.

```
sergio@vnic-ecosalmantica:/monitorizacion$ ls  
docker-compose.yml  prometheus.yml
```

También fue necesario habilitar los puertos correspondientes tanto en UFW como en la configuración de red de Oracle Cloud para permitir el acceso externo a Grafana y a los distintos servicios de monitorización.

```
sergio@vnic-ecosalmantica:/monitorizacion$ sudo ufw status
Status: active

To Action From
--
22/tcp ALLOW Anywhere
80/tcp ALLOW Anywhere
443/tcp ALLOW Anywhere
3000/tcp ALLOW Anywhere
9090/tcp ALLOW Anywhere
25/tcp ALLOW Anywhere
22/tcp (v6) ALLOW Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)
443/tcp (v6) ALLOW Anywhere (v6)
3000/tcp (v6) ALLOW Anywhere (v6)
9090/tcp (v6) ALLOW Anywhere (v6)
```

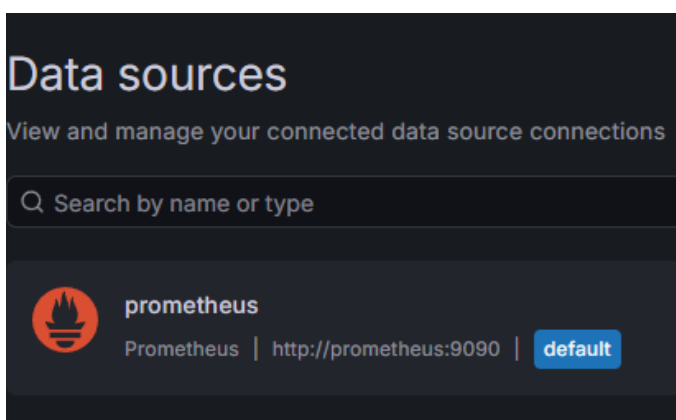
No	0.0.0.0/0	TCP	All	3000	TCP traffic for ports: 3000
☐					

Finalmente, se levantaron los contenedores mediante Docker Compose

```
sergio@vnic-ecosalmantica:/monitorizacion$ sudo docker-compose up -d
Creating network "monitorizacion_default" with the default driver
Creating node-exporter ... done
Creating prometheus ... done
Creating grafana ... done
```

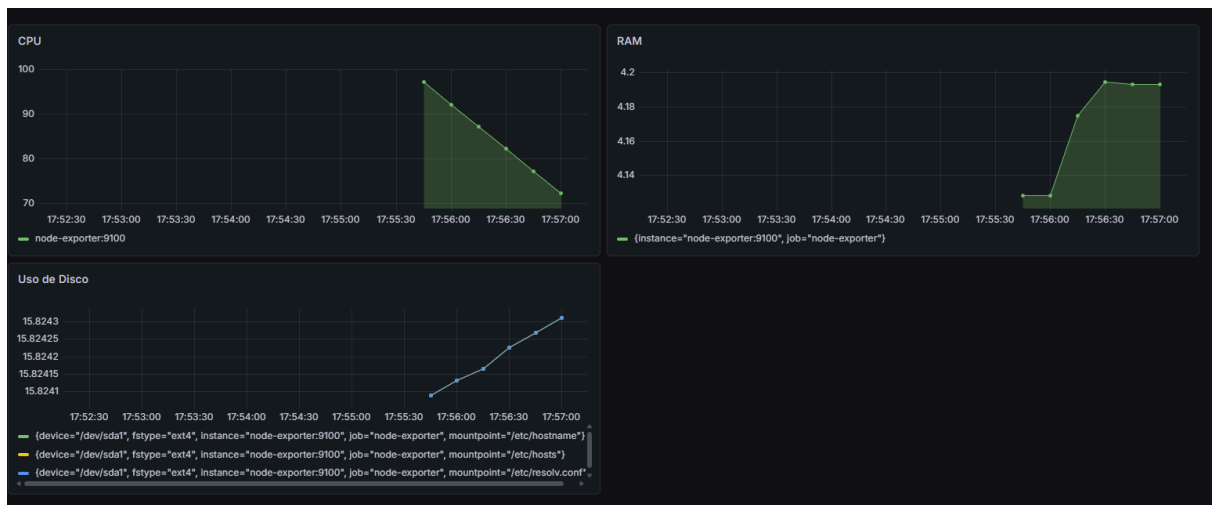
4.2. Grafana y paneles de monitorización

Una vez levantados los contenedores, se accedió a Grafana mediante el puerto 3000 del servidor para realizar la configuración inicial de la plataforma. Se conectó Grafana con Prometheus para poder mostrar todas las métricas recogidas por Node Exporter desde el propio servidor.



Con estos datos creamos un dashboard personalizado con lo que nos interesa: uso de CPU, memoria RAM y almacenamiento del disco. Gracias a este panel es posible

supervisar fácilmente el estado general del servidor y detectar posibles problemas de rendimiento.



Además, se configuró el correo del Administrador Municipal como punto de contacto dentro de Grafana para poder recibir alertas y notificaciones automáticas relacionadas con la monitorización del sistema.

The image shows the 'Notification configuration' interface in Grafana. The 'Contact points' tab is selected. The 'Update contact point' section shows the name 'AdministradorMunicipal'. The 'Integration' dropdown is set to 'Email'. The 'Addresses' field contains 'sergio@localhost'. There are buttons for 'Test', 'Duplicate', and 'Delete'. At the bottom, there are buttons for 'Save contact point' and 'Cancel'.

Para poder utilizar nuestro propio servidor de correo en el envío de alertas, fue necesario modificar la configuración SMTP de Grafana dentro del archivo docker-compose.yml. Se habilitó el soporte SMTP mediante la variable GF_SMTP_ENABLED=true y se configuró como servidor de correo host.docker.internal:25, permitiendo que el contenedor de Grafana pudiera comunicarse directamente con el servicio Postfix instalado en el propio servidor.

También se configuraron parámetros adicionales como la dirección de envío (GF_SMTP_FROM_ADDRESS), el nombre del remitente (GF_SMTP_FROM_NAME) y GF_SMTP_SKIP_VERIFY=true para evitar problemas relacionados con la verificación TLS durante las pruebas del sistema.

```

grafana:
  image: grafana/grafana-oss
  container_name: grafana
  ports:
    - "3000:3000"
  environment:
    - GF_SMTP_ENABLED=true
    - GF_SMTP_HOST=host.docker.internal:25
    - GF_SMTP_SKIP_VERIFY=true
    - GF_SMTP_FROM_ADDRESS=grafana@ecosalmantica.local
    - GF_SMTP_FROM_NAME=Grafana Ecosalmantica
    - GF_RENDERING_SERVER_URL=http://renderer:8081/render
    - GF_RENDERING_CALLBACK_URL=http://grafana:3000/
    - GF_RENDERING_RENDERER_TOKEN=ecosalmantica123
  volumes:
    - grafana-storage:/var/lib/grafana
  restart: unless-stopped

```

4.3. Informes automáticos de monitorización

Para el envío de informes diarios al Administrador municipal se implementó un sistema automático que genera un resumen del estado del servidor y lo envía por correo interno. Para ello, se desarrolló un script en Perl encargado de conectarse automáticamente a Grafana y solicitar una captura del dashboard de monitorización con las estadísticas de las últimas 24 horas. La imagen generada incluye información sobre el uso de CPU, memoria RAM y almacenamiento del sistema. Esta imagen se genera gracias al contenedor grafana-image-render que se encarga de renderizar los paneles de grafana en formato imagen. Este servicio se configuró dentro de docker-compose.yml y se conecta con grafana mediante variables de entorno y un token de autenticación.


```

- GF_RENDERING_SERVER_URL=http://renderer:8081/render
- GF_RENDERING_CALLBACK_URL=http://grafana:3000/
- GF_RENDERING_RENDERER_TOKEN=ecosalmantica123

volumes:
  - grafana-storage:/var/lib/grafana

restart: unless-stopped

renderer:

  image: grafana/grafana-image-renderer

  container_name: renderer

  environment:
    - AUTH_TOKEN=ecosalmantica123

  restart: unless-stopped

  ports:
    - "8081:8081"

```

El script Perl realiza una petición HTTP directamente a la API de renderizado de Grafana indicando el panel que se desea capturar, el intervalo de tiempo y el tamaño de la imagen. Cuando Grafana recibe la petición, envía automáticamente el dashboard al contenedor renderer, que genera la captura y devuelve la imagen final al script.

```

# Generar imagen del dashboard
my $url = "http://[REDACTED]@localhost:3000/render/d/sebdtk/informe?orgId=1&from=now-24h&to=now&width=1400&height=900&kiosk";

my $ua = LWP::UserAgent->new;
my $response = $ua->get($url);

die "Error descargando imagen\n"
  unless $response->is_success;

open(my $fh, '>', '/tmp/informe.png')
  or die "No se pudo guardar la imagen\n";

binmode($fh);

print $fh $response->decoded_content(charset => 'none');

close($fh);
# Leer imagen
open($fh, '<', '/tmp/informe.png') or die "No se pudo abrir la imagen";
binmode($fh);

my $imagen = do { local $/; <$fh > };

close($fh);

```

Una vez obtenida la imagen, el script la descarga temporalmente en el servidor y genera automáticamente un correo electrónico adjuntando el informe de monitorización. El envío se realiza utilizando el servidor Postfix configurado previamente dentro de EcoSalmántica.

```
# Crear mensaje
my $mensaje = Email::MIME->create(

    header_str => [
        From => 'grafana@ecosalmantica.local',
        To => 'sergio@localhost',
        Subject => "Informe diario Smart City - $fecha",
    ],

    parts => [

        Email::MIME->create(
            attributes => {
                content_type => "text/plain",
                charset => "UTF-8",
                encoding => "quoted-printable",
            },

            body_str => "Adjunto se encuentra el informe diario de monitorizacion.\n\nPanel Grafana:\nhttp://158.179.216.143:3000/d/sebdktk/informe",

        ),

        Email::MIME->create(
            attributes => {
                filename => "informe.png",
                content_type => "image/png",
                encoding => "base64",
                disposition => "attachment",
            },

            body => $imagen,

        ),

    ],

);

# Envio SMTP
try {
    sendmail(
        $mensaje,
        {
            from => 'grafana@ecosalmantica.local',
            transport => Email::Sender::Transport::SMTP->new({
                host => 'localhost',
                port => 25,
            }),
        }
    );
};
```

Por motivos de seguridad, el script se configuró con permisos restringidos únicamente al usuario propietario, ya que contiene información sensible como credenciales y direcciones internas utilizadas para la comunicación con Grafana y el servidor SMTP.

```
sergio@vnic-ecosalmantica:/monitorizacion$ nano ~/informe_diario.pl
sergio@vnic-ecosalmantica:/monitorizacion$ chmod 700 ~/informe_diario.pl
```

Librerías de CPAN instaladas:

```
sergio@vnic-ecosalmantica:/monitorizacion$ sudo cpan install Email::Sender
sergio@vnic-ecosalmantica:/monitorizacion$ sudo cpan install Email::MIME
sergio@vnic-ecosalmantica:/monitorizacion$ sudo cpan install Try::Tiny
sergio@vnic-ecosalmantica:/monitorizacion$ sudo cpan install LWP::UserAgent
```

El correo llega a sergio@localhost (se supone al usuario Sergio como Administrador Municipal)

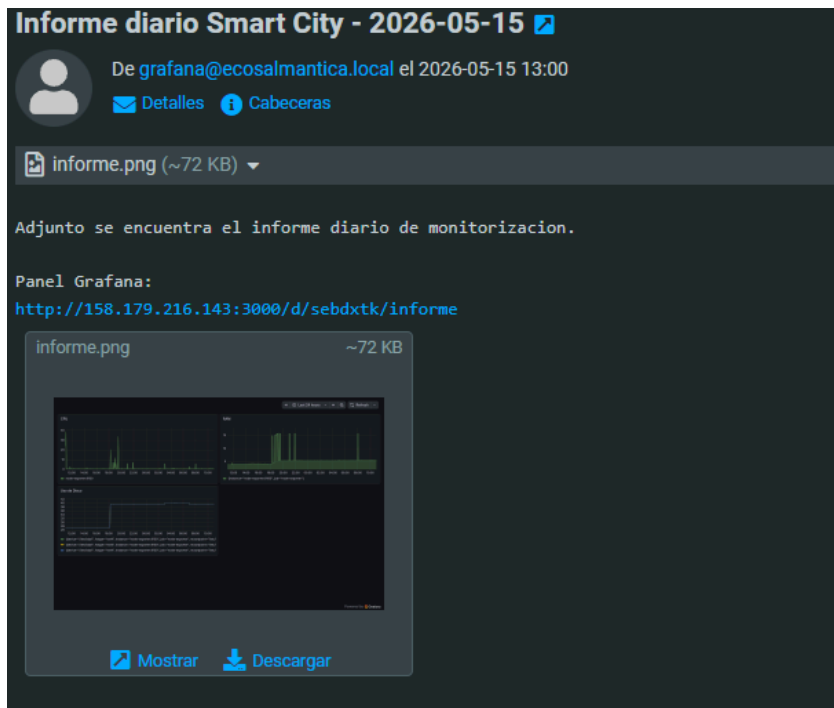
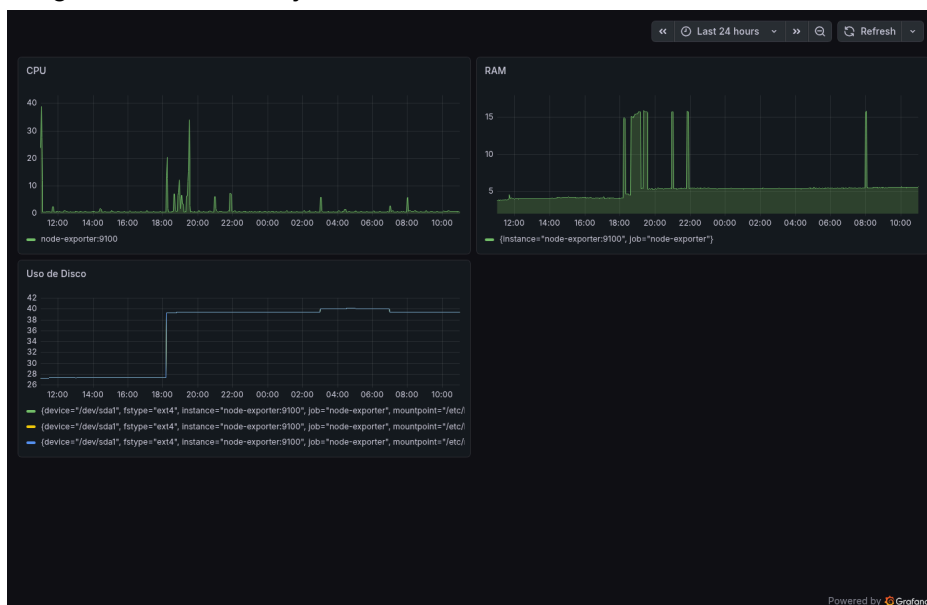


Imagen del informe adjuntada:



Finalmente, el script se añadió al crontab del sistema para ejecutarse automáticamente todos los días a las 11:00 de la mañana, permitiendo así generar y enviar informes diarios de monitorización sin intervención manual.

```
GNU nano 6.2
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
0 11 * * * /usr/bin/perl /home/sergio/informe_diario.pl
```

5. Sistema de copias de seguridad

Se ha desarrollado un sistema automático de copias de seguridad mediante un script en Perl. Las copias generadas se almacenan directamente dentro del propio servidor, permitiendo conservar una réplica de la información y configuraciones más importantes de la plataforma.

El script se encarga de generar copias comprimidas de los directorios más importantes de la plataforma, incluyendo /home, donde se almacenan los directorios personales de los usuarios junto a sus documentos, manuales, configuraciones y webs personales.

El script está diseñado para generar archivos comprimidos de los directorios esenciales, garantizando la preservación de:

- /home: Contiene los directorios personales de los usuarios, incluyendo documentos, manuales, configuraciones y sitios web personales.
- /etc: Almacena la configuración fundamental del sistema operativo y de todos los servicios instalados.
- /var/www: Incluye toda la plataforma web que corre bajo Apache.
- /var/log: Preserva los registros de actividad del sistema y la seguridad.
- /root: Guarda scripts administrativos y tareas de automatización críticas.
- /monitorizacion: Contiene los archivos de configuración de herramientas como Docker, Prometheus y Grafana.

El sistema también realiza una copia de seguridad automática de la base de datos MariaDB, asegurando la conservación de información persistente clave como usuarios, publicaciones, sesiones e incidencias, entre otros.

```
sub add_dir {
    my ($dir) = @_ ;

    find(
        sub {
            return unless -f $_;
            $star->add_files($File::Find::name);
        },
        $dir
    );
}

add_dir("/home");
add_dir("/etc");
add_dir("/var/www");
add_dir("/var/log");
add_dir("/root");
add_dir("/monitorizacion");

$star->write(
    "$backup_dir/sistema_`$fecha`.tar.gz",
    COMPRESS_GZIP
);
```

Además, el sistema implementa una política automática de rotación de backups para evitar un crecimiento excesivo del almacenamiento utilizado por las copias de seguridad. Para ello, el script comprueba la antigüedad de los archivos almacenados dentro del directorio /backups y elimina automáticamente aquellas copias con más de 7 días de antigüedad. De esta manera, el sistema mantiene únicamente los backups más recientes, permitiendo conservar un histórico limitado de copias sin ocupar espacio innecesario en el servidor.

```
find(
    sub {

        return unless -f $_;

        my $dias =
            (time - (stat($_))[9]) / 86400;

        if ($dias > 7) {
            unlink($_);
        }
    },
    $backup_dir
);
```

La configuración del script de copia de seguridad implementa permisos restrictivos, permitiendo el acceso solo al usuario root. Esta medida se adoptó como precaución

de seguridad para impedir que otros usuarios del sistema puedan acceder a la información confidencial contenida en las copias de seguridad.

```
sergio@vnic-ecosalmantica:~$ sudo chmod 700 /root/backup_sistema.pl
```

Finalmente, el script se añadió al crontab del sistema para ejecutarse automáticamente todos los días a las 3:00 de la mañana. Además, la salida estándar y los posibles errores generados durante la ejecución se almacenan automáticamente en backups.log, permitiendo supervisar el correcto funcionamiento del sistema de copias de seguridad.

```
0 3 * * * /usr/bin/perl /root/backup_sistema.pl >> /var/log/backups.log 2>&1
```

Cada copia se almacena utilizando la fecha y hora de creación en el nombre del archivo, permitiendo identificar fácilmente las distintas ejecuciones automáticas realizadas por el sistema.

```
root@vnic-ecosalmantica:/backups# ls
bd_2026-05-15_03-00.sql          sistema_2026-05-14_03-00.tar.gz
sistema_2026-05-13_21-41.tar.gz sistema_2026-05-15_03-00.tar.gz
root@vnic-ecosalmantica:/backups# |
```

Librerías de CPAN utilizadas:

DBI → Utilizada para conectarse a MariaDB y acceder a la información almacenada en la base de datos desde Perl.

Archive::Tar → Encargada de generar los archivos comprimidos .tar.gz del sistema sin utilizar comandos externos de Linux

File::Find → Utilizada para recorrer automáticamente los directorios incluidos en el backup y también para localizar y eliminar copias antiguas de más de 7 días.

6. Cuotas de disco y gestión de almacenamiento

Para limitar el espacio de almacenamiento utilizado por cada usuario dentro del servidor, se configuró un sistema de cuotas de disco sobre el sistema de archivos principal. En primer lugar, se instaló el paquete quota y posteriormente se habilitó el soporte de cuotas dentro de /etc/fstab, añadiendo la opción quota sobre la partición raíz del sistema. Tras ello, se remontó el sistema de archivos para aplicar la nueva configuración.

```
carlos@vnic-ecosalmantica:~/public_html$ sudo apt install quota
root@vnic-ecosalmantica:~# cat /etc/fstab
LABEL=cloudimg-rootfs / ext4 discard,errors=remount-ro,usrjquota=aquota.user,grpjquota=aquota.group,jqfmt=vfsv1 0 1
LABEL=UEFI /boot/efi vfat umask=0077 0 1
```

```
carlos@vnic-ecosalmantica:/$ sudo mount -o remount /
```

Durante la activación de las cuotas surgió un problema relacionado con el kernel proporcionado por Oracle Cloud, ya que este no permitía habilitar correctamente el soporte de cuotas sobre el sistema de archivos.

```
sergio@vnic-ecosalmantica:~$ uname -r  
6.8.0-1050-oracle
```

```
sergio@vnic-ecosalmantica:~$ sudo quotaon /  
quotaon: using //aquota.group on /dev/sda1 [/]: No such process  
quotaon: Quota format not supported in kernel.  
quotaon: using //aquota.user on /dev/sda1 [/]: No such process  
quotaon: Quota format not supported in kernel.
```

La solución consistió en sustituir el kernel de Oracle por un kernel genérico de Ubuntu, ya que este cambio no afectaba al funcionamiento del resto de servicios desplegados dentro del servidor. Posteriormente, se configuró GRUB para arrancar automáticamente utilizando el nuevo kernel y, tras reiniciar el sistema, se comprobó que la carga se realizaba correctamente.

```
sergio@vnic-ecosalmantica:~$ sudo apt install linux-generic
```

```
sergio@vnic-ecosalmantica:~$ sudo nano /etc/default/grub
```

```
GRUB_DEFAULT="Advanced options for Ubuntu>Ubuntu, with Linux 5.15.0-177-generic"
```

De esta manera tras reiniciar vemos cambiado el kernel:

```
sergio@vnic-ecosalmantica:~$ uname -r  
5.15.0-177-generic
```

Una vez solucionado el problema, se activaron las cuotas del sistema y se configuraron límites automáticos para los nuevos usuarios registrados dentro de la plataforma.

```
sergio@vnic-ecosalmantica:~$ sudo quotaon /  
sergio@vnic-ecosalmantica:~$ sudo quotaon -p /  
group quota on / (/dev/sda1) is on  
user quota on / (/dev/sda1) is on  
project quota on / (/dev/sda1) is off
```

Tras registrar y activar un usuario desde la plataforma web, se comprobó que las cuotas se asignaban correctamente de forma automática. En la imagen puede verse cómo el sistema aplica al usuario los límites de almacenamiento configurados previamente dentro de `activacion_usuarios.pl`, estableciendo una cuota blanda cercana a 40 MB y una cuota dura próxima a 50 MB.

```
sergio@vnic-ecosalmantica:~$ sudo quota -u pedro
Disk quotas for user pedro (uid 1010):
    Filesystem  blocks    quota   limit   grace    files   quota   limit   grace
    /dev/sda1      32  40960  51200         8        0        0
```

7. Seguridad y registros del sistema

Con el objetivo de mejorar la seguridad y mantener un control sobre la actividad realizada dentro del servidor, se implementó un sistema de logs encargado de registrar tanto las conexiones SSH como los accesos realizados desde la plataforma web dentro de un mismo archivo de registro.

Gracias a este sistema es posible almacenar información sobre usuarios, direcciones IP, fecha y hora de acceso, así como intentos correctos y fallidos de autenticación. Esto permite mantener trazabilidad sobre la actividad del servidor y detectar posibles accesos no autorizados o conexiones sospechosas desde Internet.

7.1. Registro de accesos SSH/SFTP

Para poder registrar los accesos SSH realizados al servidor, se instaló y configuró rsyslog, servicio encargado de gestionar los registros del sistema en Ubuntu Server. Gracias a ello, el servidor genera automáticamente archivos como auth.log, donde se almacenan los eventos relacionados con autenticación y conexiones SSH.

```
sergio@vnic-ecosalmantica:~$ sudo apt install rsyslog

sergio@vnic-ecosalmantica:~$ sudo systemctl enable --now rsyslog

sergio@vnic-ecosalmantica:~$ sudo systemctl status rsyslog
● rsyslog.service - System Logging Service
   Loaded: loaded (/lib/systemd/system/rsyslog.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-05-14 14:26:39 UTC; 35min ago

sergio@vnic-ecosalmantica:~$ sudo cat /var/log/auth.log
May 14 14:23:41 vnic-ecosalmantica sshd[37367]: Failed password for root from 45.148.10.152 port 3138 ssh2
May 14 14:23:41 vnic-ecosalmantica sshd[37367]: Failed password for root from 45.148.10.152 port 3138 ssh2
May 14 14:23:44 vnic-ecosalmantica sshd[37367]: Failed password for root from 45.148.10.152 port 3138 ssh2
May 14 14:23:45 vnic-ecosalmantica sshd[37367]: Connection reset by authenticating user root 45.148.10.152 port 3138 [preauth]
May 14 14:23:44 vnic-ecosalmantica sshd[37367]: Failed password for root from 45.148.10.152 port 3138 ssh2
May 14 14:23:45 vnic-ecosalmantica sshd[37367]: Connection reset by authenticating user root 45.148.10.152 port 3138 [preauth]
May 14 14:23:45 vnic-ecosalmantica sshd[37367]: PAM 4 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser=
rhost=45.148.10.152 user=root
May 14 14:23:45 vnic-ecosalmantica sshd[37367]: PAM service(sshd) ignoring max retries; 5 > 3
May 14 14:23:45 vnic-ecosalmantica sshd[37367]: PAM 4 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser=
```

A partir de este archivo se desarrolló un script en Perl encargado de monitorizar continuamente los accesos al servidor y generar un log propio para EcoSalmántica.

Después, se creó el archivo `/var/log/ecosalmantica_accesos.log`, utilizado para almacenar todos los registros generados por el sistema de monitorización. Posteriormente, se configuraron los permisos necesarios para permitir la escritura desde los scripts utilizados por la plataforma.

El script `monitor_auth.pl` se almacenó dentro del directorio `/opt/ecosalmantica/scripts` y se configuró con permisos restrictivos para permitir su ejecución únicamente al usuario propietario.

```
sergio@vnic-ecosalmantica:~$ sudo touch /var/log/ecosalmantica_accesos.log
sergio@vnic-ecosalmantica:~$ sudo chown root:www-data /var/log/ecosalmantica_accesos.log
sergio@vnic-ecosalmantica:~$ sudo chmod 664 /var/log/ecosalmantica_accesos.log

sergio@vnic-ecosalmantica:~$ sudo nano /opt/ecosalmantica/scripts/monitor_auth.pl
sergio@vnic-ecosalmantica:~$ sudo chmod 700 /opt/ecosalmantica/scripts/monitor_auth.pl
sergio@vnic-ecosalmantica:~$ sudo chown root:root /opt/ecosalmantica/scripts/monitor_auth.pl
```

En la imagen puede observarse parte del script, donde se analizan las líneas de `auth.log` buscando autenticaciones SSH correctas y fallidas para registrar automáticamente el usuario, dirección IP y tipo de acceso detectado.

```
if ($line =~ /Accepted .* for (\w+) from ([0-9.]+)/) {
    $evento =
        "[" . scalar(localtime) . "] SSH_OK usuario=$1 ip=$2\n";
} elsif (
    $line =~ /authentication failure.*rhost=([0-9.]+).*user=(\w+)/
) {
    $evento =
        "[" . scalar(localtime) . "] SSH_FAIL usuario=$2 ip=$1\n";
}
```

Para mantener el sistema de monitorización funcionando de forma permanente, el script `monitor_auth.pl` se configuró como un servicio `systemd` denominado `monitor-auth.service`. Gracias a ello, el monitor de accesos se inicia automáticamente durante el arranque del servidor y permanece ejecutándose continuamente en segundo plano analizando en tiempo real las nuevas entradas añadidas a `auth.log`.

```

root@vnic-ecosalmantica:/opt/ecosalmantica/scripts# systemctl status monitor-auth.service
● monitor-auth.service - Monitor de accesos SSH Ecosalmantica
   Loaded: loaded (/etc/systemd/system/monitor-auth.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-05-14 14:56:29 UTC; 23h ago
     Main PID: 43597 (perl)
       Tasks: 1 (limit: 28654)
      Memory: 1.1M
         CPU: 3.108s
    CGroup: /system.slice/monitor-auth.service
            └─43597 /usr/bin/perl /opt/ecosalmantica/scripts/monitor_auth.pl

May 14 14:56:29 vnic-ecosalmantica systemd[1]: Started Monitor de accesos SSH Ecosalmantica.
root@vnic-ecosalmantica:/opt/ecosalmantica/scripts#

```

7.2. Registro de accesos web

Además de las conexiones SSH, también se implementó un sistema de registro para los accesos realizados desde la plataforma web de EcoSalmántica.

Cada vez que un usuario inicia sesión correctamente desde la web mediante los scripts CGI de autenticación, el sistema almacena automáticamente información como el usuario utilizado, dirección IP, fecha y hora de acceso dentro del mismo archivo `/var/log/ecosalmantica_accesos.log`.

```

registrar_log("WEB_OK", $usuario);

unless (es_valida($usuario, $password)) {
    registrar_log("WEB_FAIL", $usuario);
}

sub registrar_log {
    my ($tipo, $usuario) = @_;

    my $ip = $ENV{REMOTE_ADDR} || 'desconocida';

    if (open(my $log, '>>', '/var/log/ecosalmantica_accesos.log')) {
        print $log
            "[" . scalar(localtime) . " ] " .
            "$tipo usuario=$usuario ip=$ip\n";
        close($log);
    }
}

```

Gracias a ello, es posible mantener un registro centralizado tanto de los accesos SSH como de las autenticaciones realizadas desde la plataforma web, facilitando la trazabilidad y supervisión de la actividad del sistema.

En la imagen puede observarse el funcionamiento del sistema de registros unificado:

```

root@vnic-ecosalmantica:/var/log# cat ecosalmantica_accesos.log
[Thu May 14 14:57:10 2026] SSH_OK usuario=sergio ip=79.116.220.51
[Thu May 14 14:57:41 2026] SSH_FAIL usuario=sergio ip=79.116.220.51
[Thu May 14 14:57:43 2026] SSH_FAIL usuario=root ip=2.57.122.195
[Thu May 14 14:58:01 2026] SSH_FAIL usuario=root ip=2.57.122.195
[Thu May 14 14:58:09 2026] WEB_FAIL usuario=de ip=79.116.220.51
[Thu May 14 14:59:43 2026] SSH_FAIL usuario=root ip=92.118.39.23
[Thu May 14 14:59:58 2026] SSH_FAIL usuario=root ip=92.118.39.23
[Thu May 14 15:01:43 2026] SSH_FAIL usuario=root ip=45.148.10.152
[Thu May 14 15:02:00 2026] SSH_FAIL usuario=root ip=45.148.10.152
[Thu May 14 15:03:44 2026] SSH_FAIL usuario=root ip=45.148.10.147
[Thu May 14 15:04:01 2026] SSH_FAIL usuario=root ip=45.148.10.147
[Thu May 14 15:05:46 2026] SSH_FAIL usuario=root ip=2.57.122.194
[Thu May 14 15:06:02 2026] SSH_FAIL usuario=root ip=2.57.122.194
[Thu May 14 15:07:46 2026] SSH_FAIL usuario=root ip=2.57.122.196
[Thu May 14 15:08:00 2026] SSH_FAIL usuario=root ip=2.57.122.196
[Thu May 14 15:09:47 2026] SSH_FAIL usuario=root ip=92.118.39.23
[Thu May 14 15:10:03 2026] SSH_FAIL usuario=root ip=92.118.39.23
[Thu May 14 15:11:49 2026] SSH_FAIL usuario=root ip=92.118.39.196
[Thu May 14 15:12:06 2026] SSH_FAIL usuario=root ip=92.118.39.196
[Thu May 14 15:13:49 2026] SSH_FAIL usuario=root ip=2.57.122.191
[Thu May 14 15:14:05 2026] SSH_FAIL usuario=root ip=2.57.122.191
[Thu May 14 15:15:50 2026] SSH_FAIL usuario=root ip=92.118.39.197
[Thu May 14 15:16:08 2026] SSH_FAIL usuario=root ip=92.118.39.197

```

8. Base de datos Mariadb

Para una gestión de almacenamiento robusta y escalable es necesario implementar un motor de base de datos como MariaDB, encargado de gestionar desde credenciales temporales hasta el registro de incidencias ciudadanas.

8.1 Instalación de la base de datos

La instalación se realiza mediante el gestor de paquetes del sistema:

```

carlos@vnic-ecosalmantica:/etc$ sudo apt install mariadb-server

```

8.2 Creación nueva base de datos

Una vez con el servicio en funcionamiento se accede a la consola de mariaDB para crear la base de datos del proyecto:

```

carlos@vnic-ecosalmantica:/etc$ sudo mariadb -u root

```

Creamos nueva base de datos para administrar ecosalmantica:

```

MariaDB [(none)]> CREATE DATABASE ecosalmantica;
Query OK, 1 row affected (0.000 sec)

```

En lugar de utilizar la cuenta de root para operaciones web, lo que podría ser un riesgo de seguridad, se ha creado un administrador específico llamado admineco. Este usuario tiene privilegios totales únicamente sobre la tabla ecosalmantica, queda restringido su acceso remoto del sistema de bases de datos.

```
MariaDB [(none)]> CREATE USER 'admineco'@'localhost' IDENTIFIED BY 'admineco123';
Query OK, 0 rows affected (0.001 sec)
```

```
MariaDB [(none)]> GRANT ALL PRIVILEGES ON ecosalmantica.* TO 'admineco'@'localhost';
```

Para soportar el flujo de usuarios y servicios mencionados anteriormente, se han diseñado 5 tablas:

```
MariaDB [ecosalmantica]> show tables;
+-----+
| Tables_in_ecosalmantica |
+-----+
| cambios_password         |
| clave_recuperacion       |
| incidencias              |
| tokens_verificacion      |
| usuarios                 |
+-----+
5 rows in set (0.000 sec)
```

usuarios: Almacena perfiles, roles y estados de activación:

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
login	varchar(32)	NO	UNI	NULL	
email	varchar(128)	NO	UNI	NULL	
nombre	varchar(128)	NO		NULL	
rol	enum('ciudadano', 'tecnico')	NO		ciudadano	
estado	enum('pendiente', 'confirmado', 'activo', 'baja', 'eliminado')	NO		pendiente	
fecha_alta	datetime	YES		current_timestamp()	
pass_tmp	varchar(255)	YES		NULL	
fecha_registro	timestamp	NO		current_timestamp()	
fecha_baja	datetime	YES		NULL	

tokens_verificacion: Gestiona los códigos temporales de 5 dígitos para el alta de cuentas:

```
MariaDB [ecosalmantica]> describe tokens_verificacion;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)   | NO   | PRI | NULL    | auto_increment |
| usuario_id | int(11)   | NO   | MUL | NULL    |              |
| token      | varchar(64) | NO   | UNI | NULL    |              |
| expira     | datetime  | NO   |     | NULL    |              |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.001 sec)
```

incidencias: Almacena los avisos reportados por los ciudadanos:

```
MariaDB [ecosalmantica]> describe incidencias;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
titulo	varchar(200)	YES		NULL	
descripcion	text	YES		NULL	
categoria	varchar(100)	YES		NULL	
urgencia	varchar(50)	YES		NULL	
departamento	varchar(100)	YES		NULL	
fecha	timestamp	NO		current_timestamp()	

```
7 rows in set (0.001 sec)
```

cambios_password: Auditoría de modificaciones en las credenciales de acceso:

```
MariaDB [ecosalmantica]> describe cambios_password;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
login	varchar(32)	NO		NULL	
hash_nuevo	varchar(128)	NO		NULL	
fecha	datetime	YES		current_timestamp()	

```
4 rows in set (0.001 sec)
```

clave_recuperacion: Almacena los hashes de seguridad para restablecer cuentas

```
MariaDB [ecosalmantica]> describe clave_recuperacion;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
usuario_id	int(11)	NO	MUL	NULL	
clave	varchar(64)	NO	UNI	NULL	
expira	datetime	NO		NULL	
fecha_creacion	timestamp	NO		current_timestamp()	

```
5 rows in set (0.001 sec)
```

9. Servidor de correo

Para tener un canal de comunicación oficial se ha desplegado un stack completo de correo electrónico. Permite recibir notificaciones como los informes de monitorización.

9.1 Postfix

Postfix es el núcleo encargado del envío y recepción de correos. Su instalación se realiza indicando Sitio Local.

```
carlos@vnic-ecosalmantica:/etc$ sudo apt install postfix
```

```
1. No configuration 2. Internet Site 3. Internet with smarthost 4. Satellite system 5. Local only
General mail configuration type: 2
```

Le pondremos al correo un nombre:

```
System mail name: ecorreo
```

Además en el /etc/postfix/main.cf ponemos las cuotas y los límites de almacenamiento.

```
mailbox_size_limit = 4194304
message_size_limit = 3145728
```

Para que la capacidad máxima de cada buzón de correo sea de 4 MB, ya que trabaja por bytes y $4 \times 1024 \times 1024 = 4194304$
Además hacemos que el tamaño máximo de cada mensaje sea de 3MB para hacer que un solo mensaje pueda ocupar toda la capacidad del usuario.

9.2 Dovecot

Para que los ciudadanos puedan consultar sus correos, se ha instalado Dovecot, que implementa IMAP. Un protocolo que permite a los usuarios ver sus mensajes manteniendo una copia en el servidor.

```
carlos@vnic-ecosalmantica:/etc$ sudo apt install dovecot-core dovecot-imapd
```

9.3 Roundcube

Como interfaz para el ciudadano, se ha desplegado Roundcube, un cliente basado en web que ofrece una experiencia moderna.

```
carlos@vnic-ecosalmantica:/etc$ sudo apt install roundcube
```

```
Configure database for roundcube with dbconfig-common? [yes/no] yes
Please provide a password for roundcube to register with the database server. If left blank, a random password will be generated.
MySQL application password for roundcube:
Password confirmation:
```

Durante la instalación el sistema se vincula automáticamente con MariaDB

instalado previamente. Se crea una base de datos dedicada y un usuario roundcube para gestionar las preferencias y contactos de los usuarios de forma aislada.

```
carlos@vnic-ecosalmantica:/usr/lib/cgi-bin$ sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 6487
Server version: 10.6.23-MariaDB-0ubuntu0.22.04.1 Ubuntu 22.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| ecosalmantica |
| information_schema |
| mysql |
| performance_schema |
| roundcube |
| sys |
+-----+
6 rows in set (0.000 sec)

MariaDB [(none)]> |
```

10. Mejoras y funcionalidades extra

10.1 Banner SSH

Como mejora adicional del servidor, se configuró un banner SSH mostrado automáticamente después del inicio de sesión de los usuarios durante las conexiones remotas al sistema.

Para ello, se eliminó el mensaje MOTD predeterminado de Ubuntu Server y se configuró un nuevo mensaje.

Desactivamos el banner predeterminado

```
sergio@vnic-ecosalmantica:~$ sudo chmod -x /etc/update-motd.d/*
```

Creamos el banner propio y probamos su funcionamiento

```
sergio@vnic-ecosalmantica:~$ sudo nano /etc/update-motd.d/99-ecosalmantica
sergio@vnic-ecosalmantica:~$ sudo chmod +x /etc/update-motd.d/99-ecosalmantica
sergio@vnic-ecosalmantica:~$ run-parts /etc/update-motd.d/

ECOSALMANTICA

SMART CITY PLATFORM
```

Después de activarlo se verá tras hacer login por SSH

```
PS C:\Users\SERGI> ssh sergio@ecosalmantica1.duckdns.org
(sergio@ecosalmantica1.duckdns.org) Password:

ECOSALMANTICA

SMART CITY PLATFORM

You have mail.
Last login: Thu May 14 15:45:31 2026 from 79.116.220.51
sergio@vnic-ecosalmantica:~$ |
```

10.2. Clasificación automática de incidencias mediante IA

Se ha integrado el modelo Llama 3.2 mediante Ollama para clasificar automáticamente las incidencias enviadas por los ciudadanos desde la web.

En primer lugar, se instaló Ollama dentro del servidor Ubuntu y posteriormente se descargó el modelo llama3.2.

```
sergio@vnic-ecosalmantica:~$ curl -fsSL https://ollama.com/install.sh | sh
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-arm64.tar.zst
##### 100.0%
>>> Creating ollama user...
>>> Adding ollama user to render group...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
>>> Enabling and starting ollama service...
Created symlink /etc/systemd/system/default.target.wants/ollama.service → /etc/systemd/system/ollama.service.
WARNING: Unable to detect NVIDIA/AMD GPU. Install lspci or lshw to automatically detect and install GPU dependencies.

sergio@vnic-ecosalmantica:~$ ollama run llama3.2
pulling manifest
pulling dde5aa3fc5ff: 100%
pulling 966de95ca8a6: 100%
pulling fcc5a6bec9da: 100%
pulling a70ff7e570d9: 100%
pulling 56bb8bd477a5: 100%
pulling 34bb5ab01051: 100%
verifying sha256 digest
writing manifest
success
```

Una vez comprobado el funcionamiento del modelo desde terminal, la IA se integró dentro de la plataforma web mediante el CGI incidencia.pl, desarrollado en Perl.

Sección dentro de [portal.pl](#) para redirigir a incidencia.pl:

Servicios Disponibles

- [Correo municipal](#)
- [Blog ciudadano](#)
- [Web personal](#)
- [FTP/SFTP seguro](#)
- [Mi perfil](#)
- [Nueva incidencia municipal IA](#)
- [Gestión de manuales técnicos](#)

El script genera automáticamente un prompt donde se definen las categorías, niveles de urgencia y departamentos municipales permitidos. A partir de la descripción escrita por el ciudadano, el CGI envía automáticamente el texto al modelo llama3.2 mediante Ollama para que la IA analice el contenido de la incidencia y genere información relacionada con la categoría del problema, el nivel de urgencia y el departamento municipal encargado de gestionarlo.

```
sergio@vnic-ecosalmantica:~$ sudo nano /usr/lib/cgi-bin/incidencia.pl
```

Prompt para Ollama dentro del script:

```
Clasifica la siguiente incidencia municipal.

SOLO puedes usar:

Categorías:
- Alumbrado
- Limpieza
- Seguridad
- Trafico
- Agua
- Parques

Urgencia:
- Baja
- Media
- Alta

Departamentos:
- Servicios Técnicos
- Policía
- Medio Ambiente
- Movilidad Urbana

Devuelve EXACTAMENTE este formato:

Categoría: ...
Urgencia: ...
Departamento: ...

Incidencia:
"$texto"
```

La respuesta generada por Ollama es procesada desde Perl para extraer automáticamente la categoría, urgencia y departamento detectados, almacenando después toda esta información dentro de MariaDB junto al resto de datos de la incidencia.

```

ENV{'HOME'} = '/opt/ollama-web';

my $pid = open2(
    my $out,
    my $in,
    '/usr/local/bin/ollama',
    'run',
    'llama3.2'
);

print $in $prompt;

close($in);

my $respuesta = do {
    local $/;
    decode('UTF-8', <$out>);
};

waitpid($pid, 0);

my ($categoria) =
    $respuesta =~ /Categoria:\s*([^\n\r]+)/i;

my ($urgencia) =
    $respuesta =~ /Urgencia:\s*([^\n\r]+)/i;

my ($departamento) =
    $respuesta =~ /Departamento:\s*([^\n\r]+)/i;

$categoria    ||= 'Sin clasificar';
$urgencia      ||= 'Media';
$departamento ||= 'Servicios Técnicos';

```

Creamos nueva tabla en la base de datos para guardar las incidencias:

```

MariaDB [ecosalmantica]> CREATE TABLE incidencias (
    id INT AUTO_INCREMENT PRIMARY KEY,    titulo VARCHAR(200),
    descripcion TEXT, categoria VARCHAR(100), urgencia VARCHAR(50), departamento VARCHAR(100), fecha
    TIMESTAMP DEFAULT CURRENT_TIMESTAMP );

```

```

MariaDB [ecosalmantica]> SHOW TABLES;

```

```

+-----+
| Tables_in_ecosalmantica |
+-----+
| cambios_password         |
| clave_recuperacion       |
| incidencias              |
| tokens_verificacion      |
| usuarios                 |
+-----+

```

```

5 rows in set (0.000 sec)

```

```

MariaDB [ecosalmantica]>

```

```

MariaDB [ecosalmantica]> SELECT * FROM incidencias;

```

id	titulo	descripcion	categoria	urgencia	departamento	fecha
1	Pelea	Se estan peleando 5 personas en la plaza mayor	Seguridad	Alta	Policía	2026-05-14 19:06:04
2	Carril bici	Me gustaria que pongan mas carril bici y lo dejen bonito	Parques	Baja	Medio Ambiente	2026-05-14 19:06:29

```

2 rows in set (0.000 sec)

```

```

MariaDB [ecosalmantica]>

```

Al principio surgieron problemas ya que www-data no podía ejecutar correctamente la IA desde los scripts CGI, por ello el CGI devolvió continuamente los valores por defecto para los campos a clasificar.

```
sergio@vnic-ecosalmantica:~$ sudo -u www-data /usr/local/bin/ollama run llama3.2
Error: mkdir /var/www/.ollama: permission denied
```

Para solucionar este problema, se creó un directorio específico y aislado para la ejecución de Ollama desde Apache, permitiendo separar la configuración utilizada por el servicio web del entorno del usuario principal del sistema.

```
sergio@vnic-ecosalmantica:~$ sudo mkdir -p /opt/ollama-web
sergio@vnic-ecosalmantica:~$ sudo chown www-data:www-data /opt/ollama-web
```

Posteriormente, dentro del CGI incidencia.pl se configuró la variable de entorno HOME apuntando al nuevo directorio creado para Ollama, permitiendo que el modelo pudiera ejecutarse correctamente desde Apache mediante el usuario www-data.

```
$ENV{'HOME'} = '/opt/ollama-web';
```

Aquí se pueden ver los paneles de la plataforma web con los que interactúa el usuario para enviar incidencias municipales, así como la respuesta generada automáticamente por la IA mostrando la categoría detectada, el nivel de urgencia y el departamento municipal asignado.

Nueva incidencia municipal

Las incidencias serán clasificadas automáticamente mediante IA municipal.

Título

Farolas apagadas en avenida principal

Descripción de la incidencia

Desde hace varios días varias farolas de la Avenida de Portugal permanecen apagadas durante la noche, especialmente en el tramo cercano al parque infantil. La zona tiene muy poca visibilidad y supone un riesgo para peatones y vehículos.

Enviar incidencia

Sistema Inteligente Ecosalmantica

La incidencia será clasificada automáticamente por IA:

- Categoría
- Urgencia
- Departamento responsable

Volver al portal

Incidencia registrada correctamente

Título:

Farolas apagadas en avenida principal

Descripción:

Desde hace varios días varias farolas de la Avenida de Portugal permanecen apagadas durante la noche, especialmente en el tramo cercano al parque infantil. La zona tiene muy poca visibilidad y supone un riesgo para peatones y vehículos.

Categoría IA:

Trafico

Urgencia IA:

Alta

Departamento IA:

Policia

Nueva incidencia

11. Conclusiones

Este trabajo nos ha dado la oportunidad de juntar muchas de las cosas vistas durante el curso y montarlas en algo que realmente funciona. Configurar un servidor con todos sus servicios, conseguir que se comuniquen entre sí y que además todo sea seguro y automático ha sido bastante más complicado de lo que parecía al principio.

Durante el desarrollo han surgido bastantes problemas, algunos bastante frustrantes, como el tema del kernel de Oracle Cloud con las cuotas de disco o conseguir que Ollama funcionara correctamente desde Apache. Aun así, al final todo termina teniendo solución y probablemente es ahí donde más hemos aprendido, entendiendo realmente por qué ocurrían los errores y cómo resolverlos.

En cuanto a la seguridad, hemos intentado razonar cada decisión y pensar dónde debía ir cada cosa y por qué. Evidentemente el sistema no es perfecto y seguramente haya aspectos mejorables, pero la idea principal ha sido intentar que cada parte del proyecto tuviera sentido y estuviera hecha siguiendo una lógica real de administración y separación de permisos.

También ha sido interesante trabajar con servicios que normalmente se estudian por separado y ver cómo terminan relacionándose entre ellos dentro de un mismo entorno.

Al final, lo más importante del proyecto es haber montado algo completo y funcional desde cero, enfrentándonos a problemas reales y aprendiendo a solucionarlos. Creemos que esa parte práctica es lo que más ayuda de cara al mundo laboral y a entender realmente la administración de sistemas más allá de la teoría.